COSC2956 Internet Tools

Algoma UNIVERSITY

Building Backend Applications with Node.js, Express & Databases

What We Will Learn in This Lecture:

1-Create a backend using Node.js and Express

2-Understand how to connect to databases

3-Learn the difference between SQL and NoSQL

4-Store and retrieve data

5-Build a simple authentication system

6-Generate and use tokens (JWT)

Backend (Definition):

- The backend is the server-side part of a web application. It runs on a server, receives requests from the frontend (such as a browser or mobile app), processes the application’s business logic, interacts with databases to store or retrieve data, and then sends the appropriate response back to the client.

What is the backend responsible for?:

- Authentication (Register/Login)
- Storing and retrieving data
- Security and permissions
- Data validation
- Business logic
- API communication

See server.js

Request Flow:

- Frontend (Browser / React / Mobile App)
  ↓ HTTP Request
- Backend (Express Server)
  ↓ Query
- Database (SQL / NoSQL)
  ↓ Result
- Backend
  ↓ JSON Response
- Frontend

In other words Backend is the middle layer and front end doesn’t talk directly with the database

Why Do We Need a Database?

A backend without a database cannot remember anything.
If the server restarts, all data is lost unless it is stored permanently.

A database allows the backend to:

- Store user accounts
- Save passwords securely
- Keep application data (posts, products, orders)
- Retrieve information when requested
- Persist data even after server restarts

Two Types of Databases:

SQL(PostgreSQL)	NOSQL(MongoDB)
Uses tables with rows and columns	Uses JSON-like documents
Has a fixed schema	Flexible schema
Strong relationships using JOIN	Easy to store nested data
Good for structured systems	Good for fast development

Connecting the Backend to Databases

- **For PostgreSQL:**
- Installed PostgreSQL server
- Installed pg driver in Node.js
- Configured credentials in .env
- Created a connection pool
- Tested connection using SELECT NOW()

- **For MongoDB:**
- Installed MongoDB server
- Installed mongoose driver
- Added MONGO_URI in .env
- Connected using mongoose.connect()
- Verified successful connection

PostgreSQL vs MongoDB (From a Backend Perspective):

- **PostgreSQL (SQL)**
- Requires explicit database creation
- Requires credentials (user + password)
- Uses structured tables
- Queries are written in SQL
- We tested connection using a real SQL query

- **MongoDB (NoSQL)**
- Database is created automatically when inserting data
- Local installation does not require authentication
- Uses JSON-like documents
- Connection is done through a URI string
- No query needed to verify connection

Creating a schema on postegrSQL :

```
CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
  );
```

CRUD Operations on a database:

- **CRUD** stands for:
- **C**reate → Add new data
- **R**ead → Retrieve data
- **U**pdate → Modify existing data
- **D**elete → Remove data

- **Create Student**

```
curl -X POST http://localhost:3001/students \
-H "Content-Type: application/json" \
-d '{"username":"john","password":"1234"}'
```

- **Read All Students**

```
curl http://localhost:3001/students
```

- **Read Student by ID**

```
curl http://localhost:3001/students/1
```

CRUD Operations on a database:

- **Update Student**

```
curl -X PUT http://localhost:3001/students/1 \
-H "Content-Type: application/json" \
-d '{"username":"Sam"}'
```

- **Delete Student**

```
curl -X DELETE http://localhost:3001/students/1
```